Lecture 14

EE 421 / CS 425

Digital System Design

Fall 2026

Shahid Masud

Topics

- Fractional Multiplication - Leftover
- Different Cases of Signed Multiplication in Fractional Binary Numbers

---

- Booth Encoded Multipliers
- Advantages of Booth Multipliers
- Limitations of Booth Multipliers
- STG Control for Booth Multipliers
- Examples of Booth Multiplication
- Introduce Radix-4 Multiplication

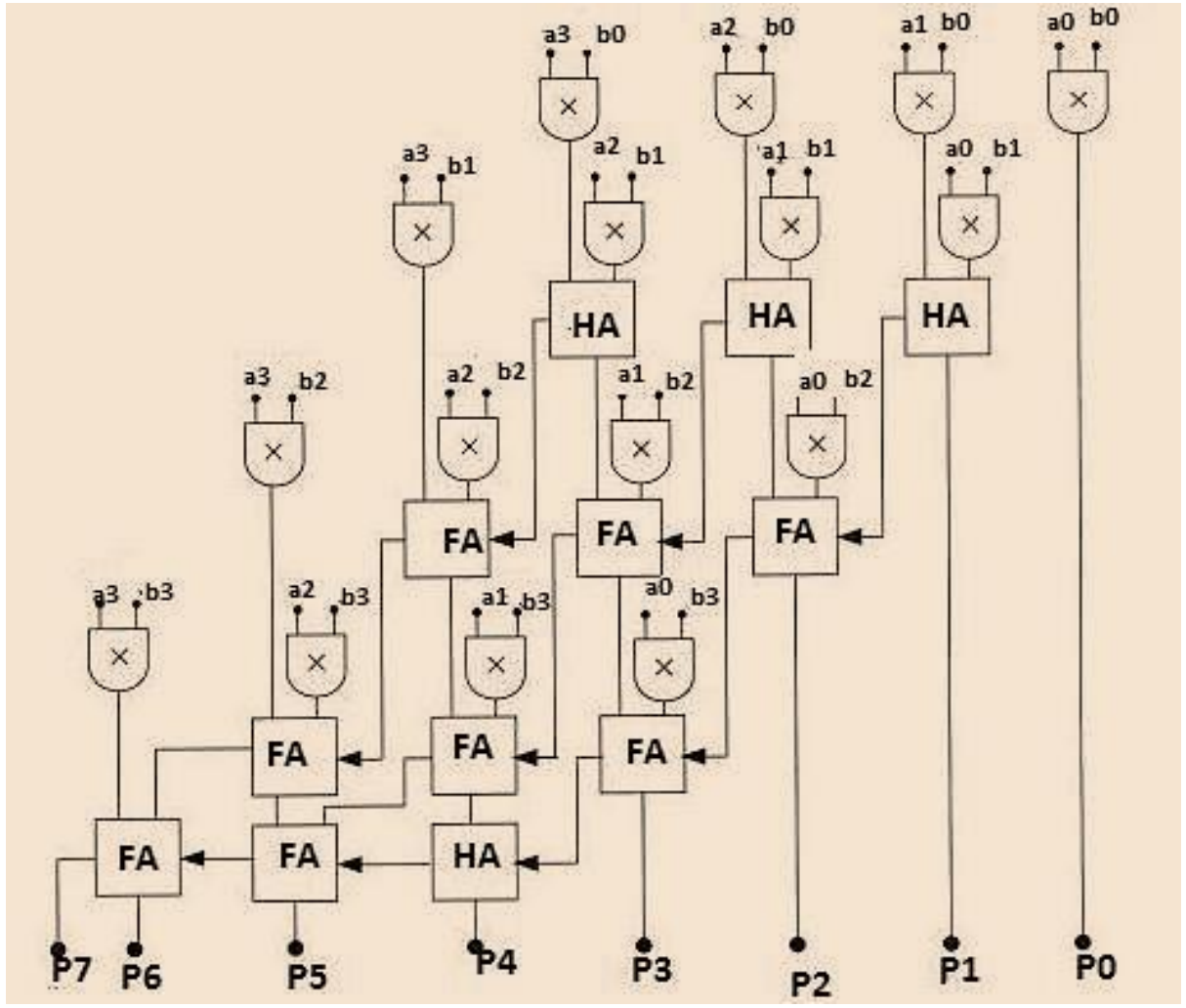
MIDTERM
Thursday 29 Oct

Recap

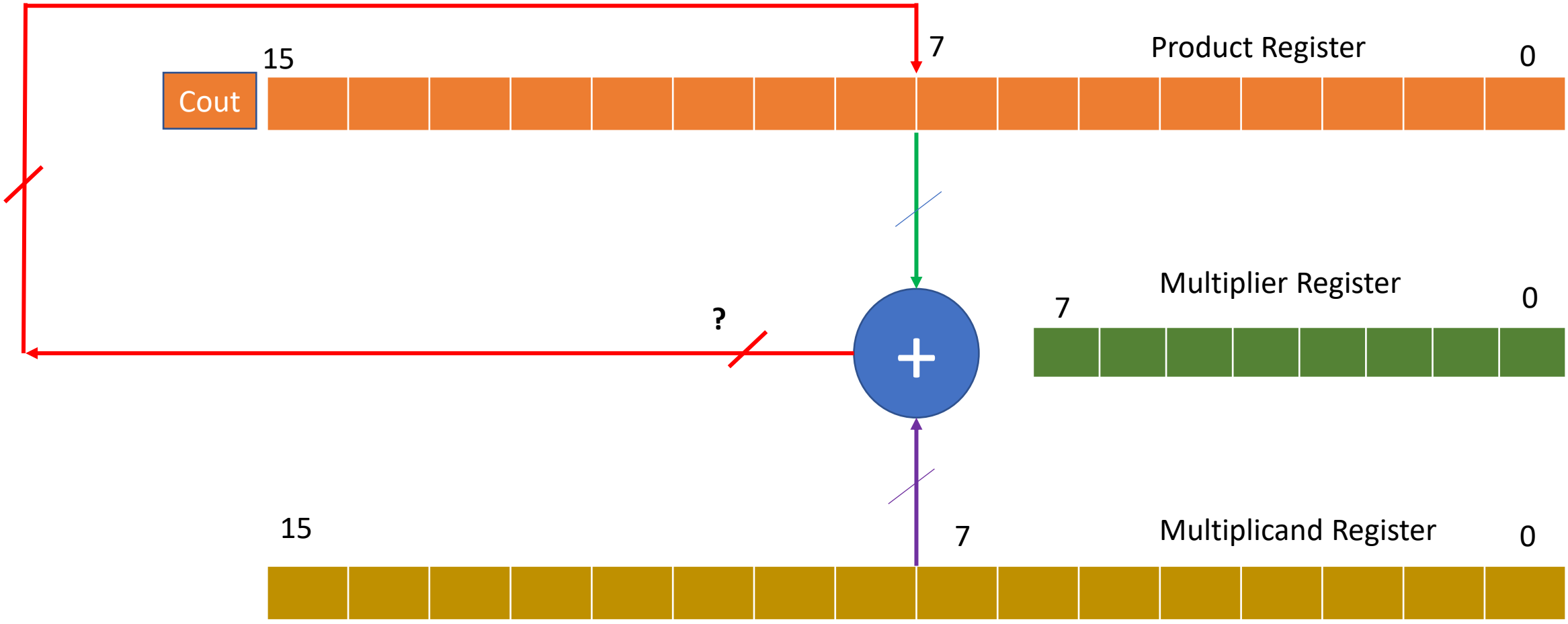
- Array Multiplier
- Pipelining and Sequential Implementation
- Datapath and Controlpath
- Reducing Registers through Shifting and Folding
- This is Circuit Related Approach

4-Bit Array Multiplier connected as AND and ADD

				A_3	A_2	A_1	A_0
				B_3	B_2	B_1	B_0
				A_3B_0	A_2B_0	A_1B_0	A_0B_0
			A_3B_1	A_2B_1	A_1B_1	A_0B_1	0
		A_3B_2	A_2B_2	A_1B_2	A_0B_2	0	0
	A_3B_3	A_2B_3	A_1B_3	A_0B_3	0	0	0
Cout	P_6	P_5	P_4	P_3	P_2	P_1	P_0

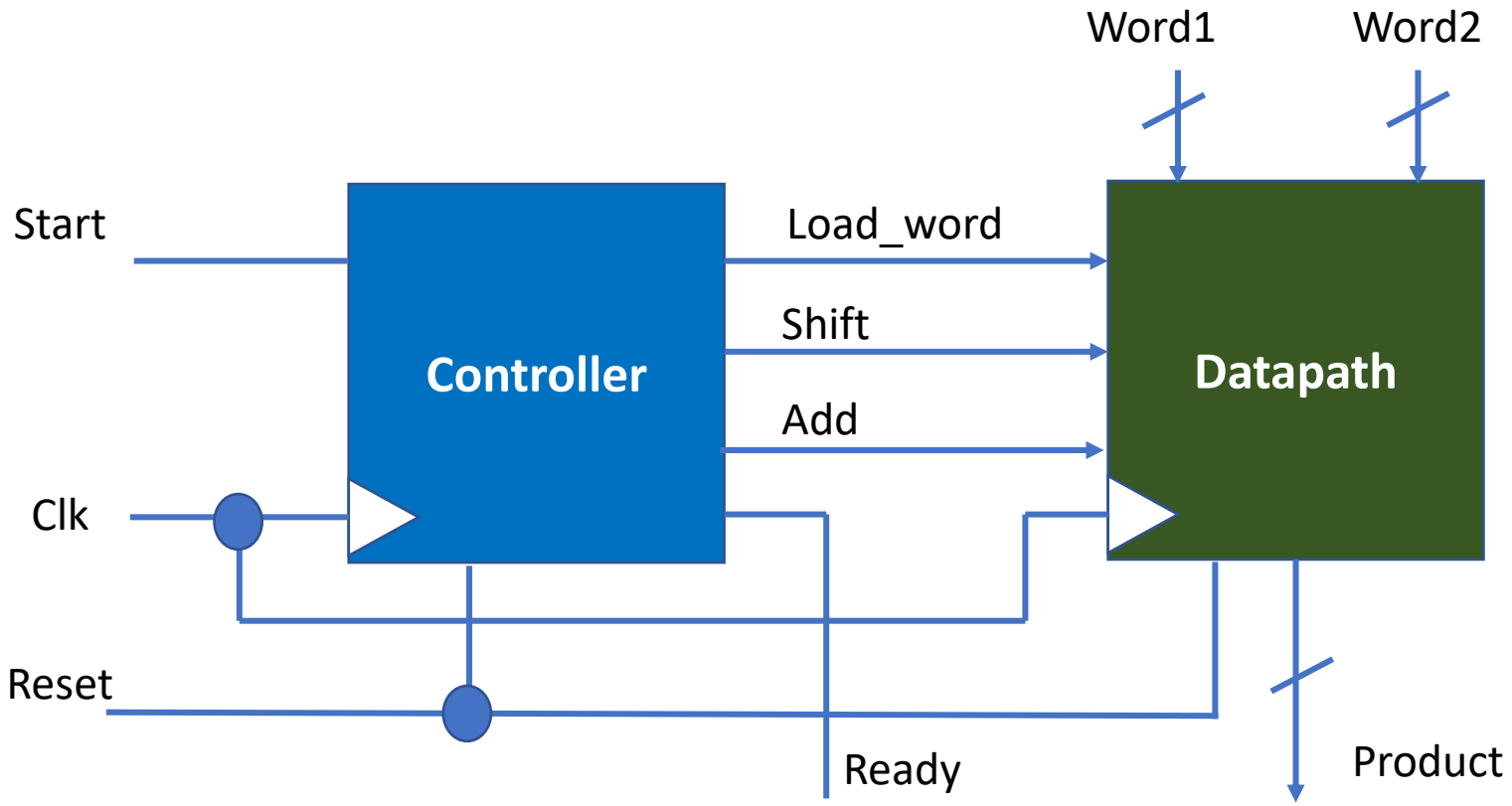


Operation of Sequential Multiplier

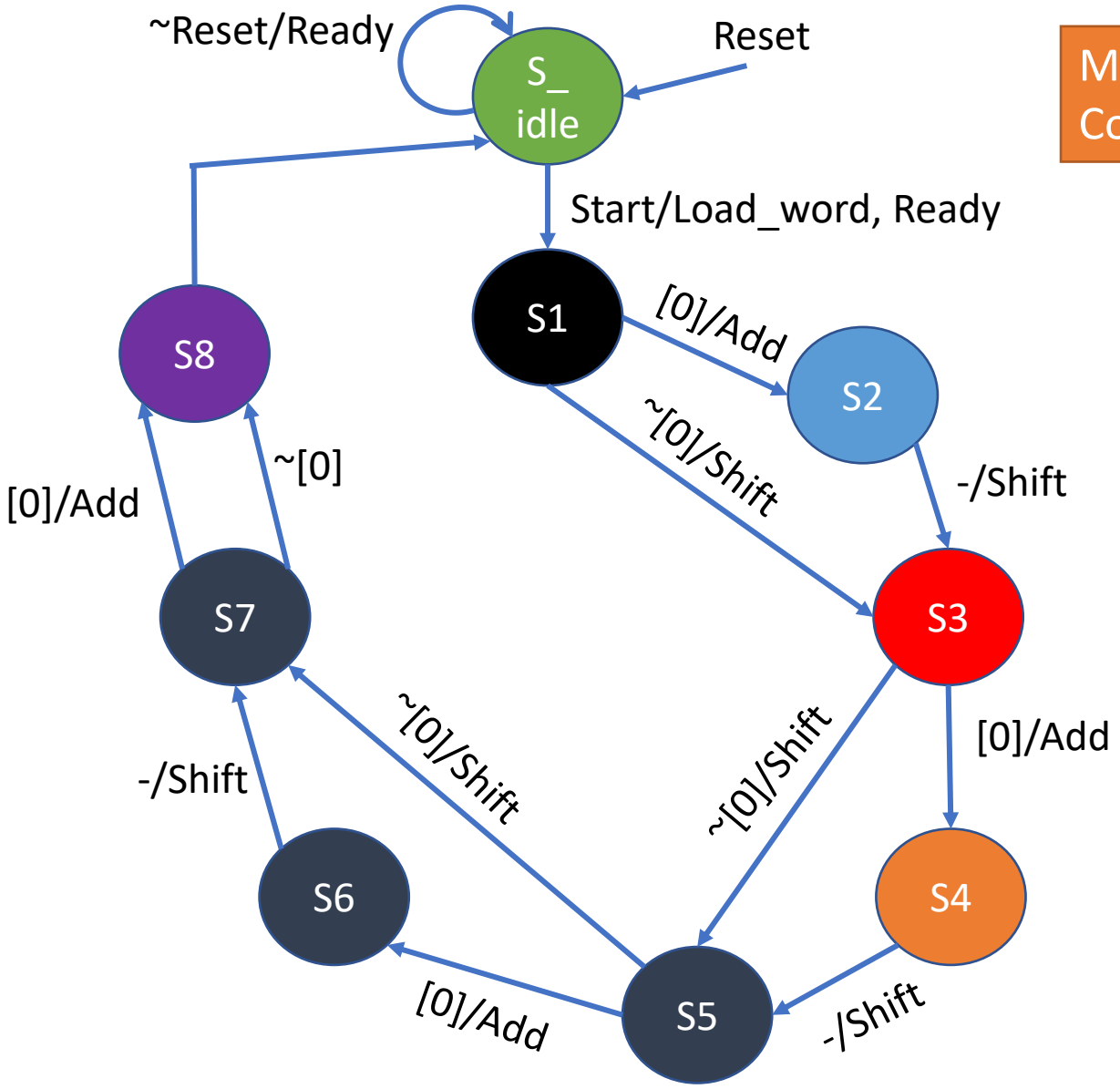


Datapath of a Sequential Multiplier

Data Path Architecture of Sequential Mult

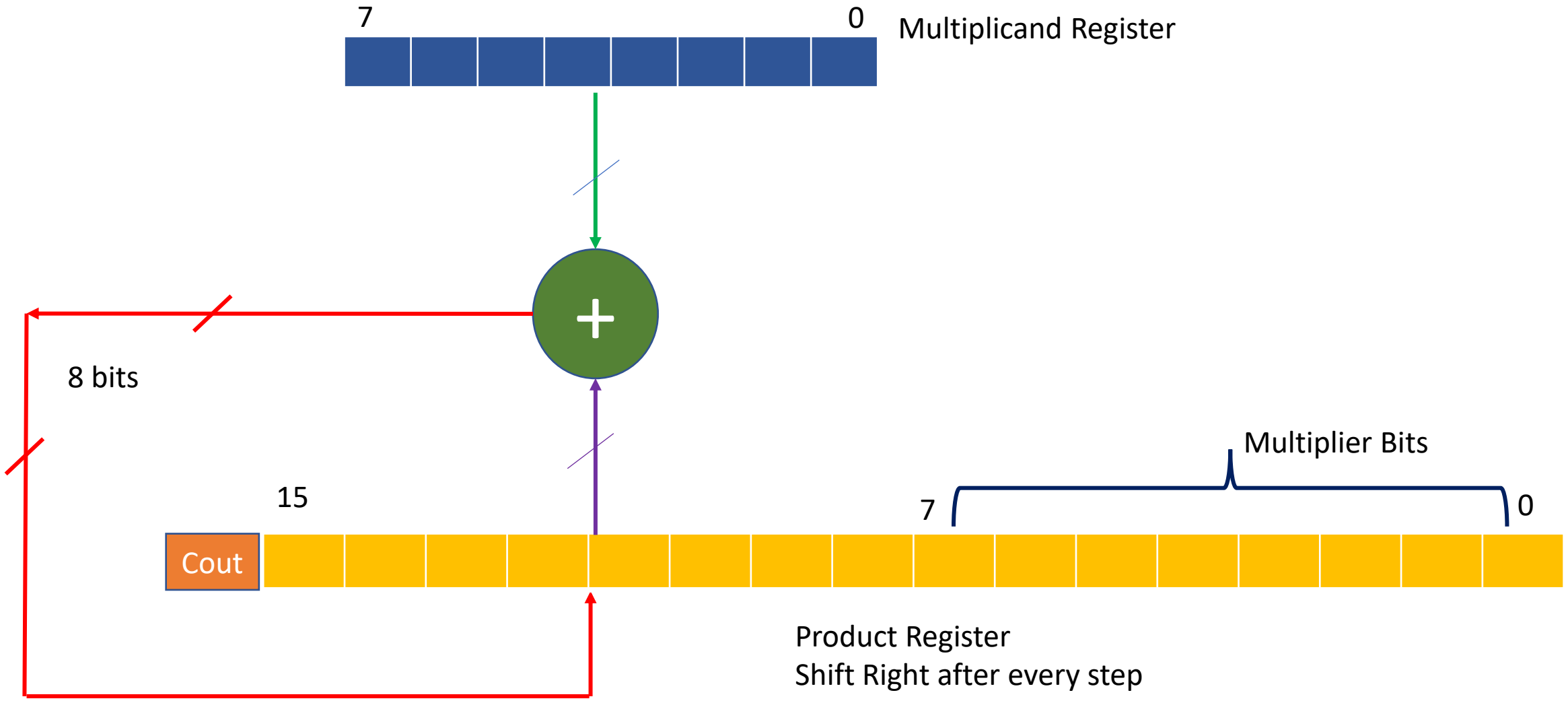


STG for a 4 Bit Sequential Binary Multiplier

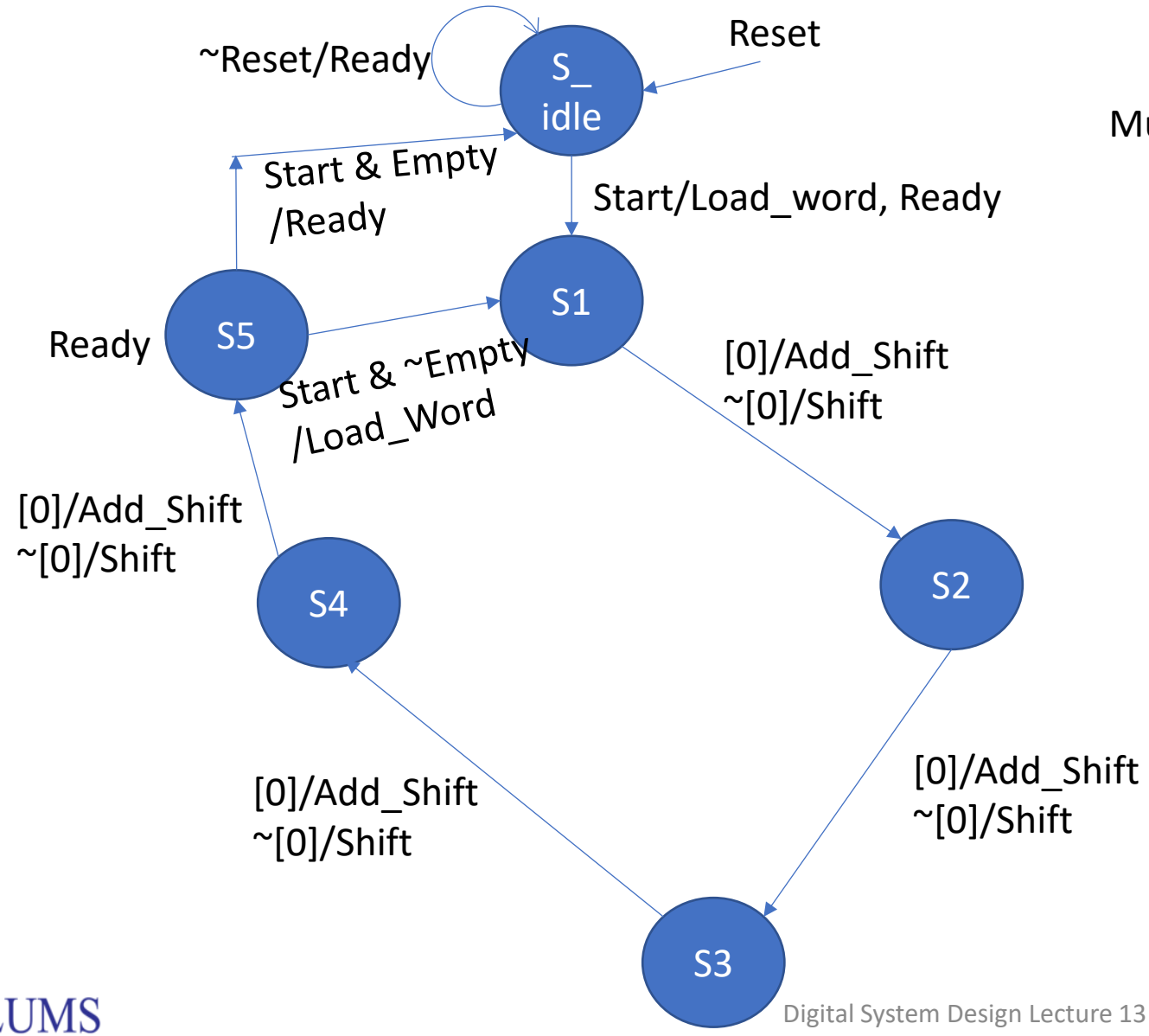


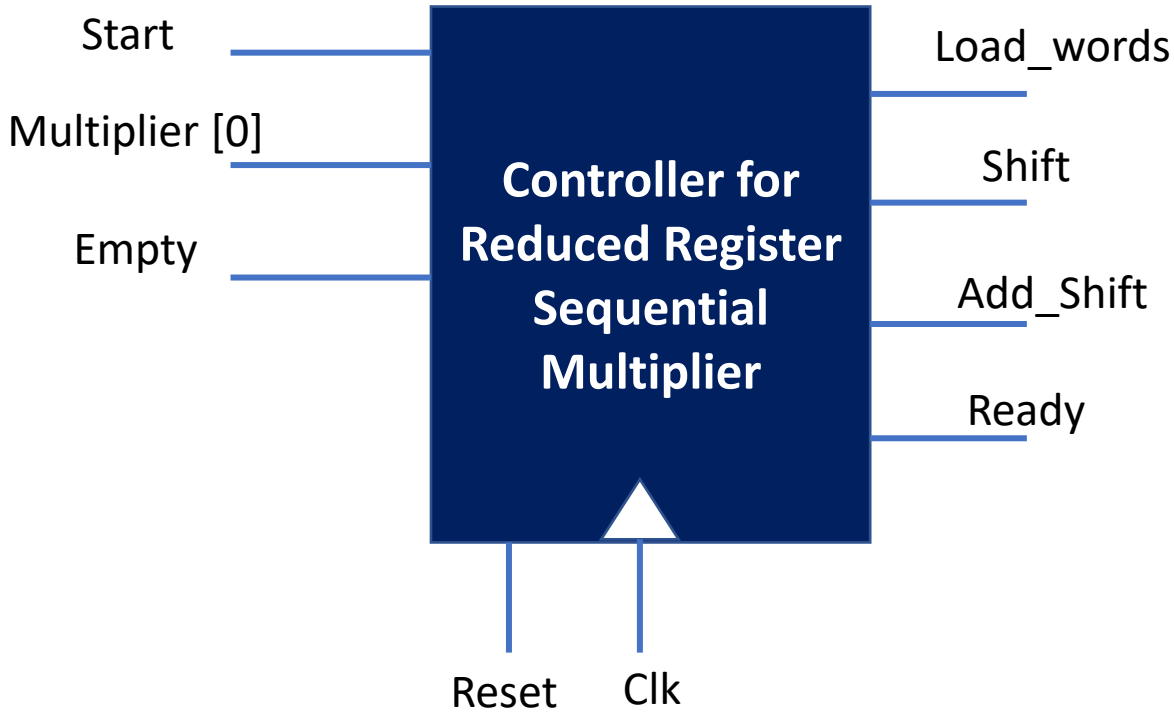
Machine returns to Idle state after Completion of 4 bit multiplication

Improvement - Sequential Multiplier with Reduced Registers

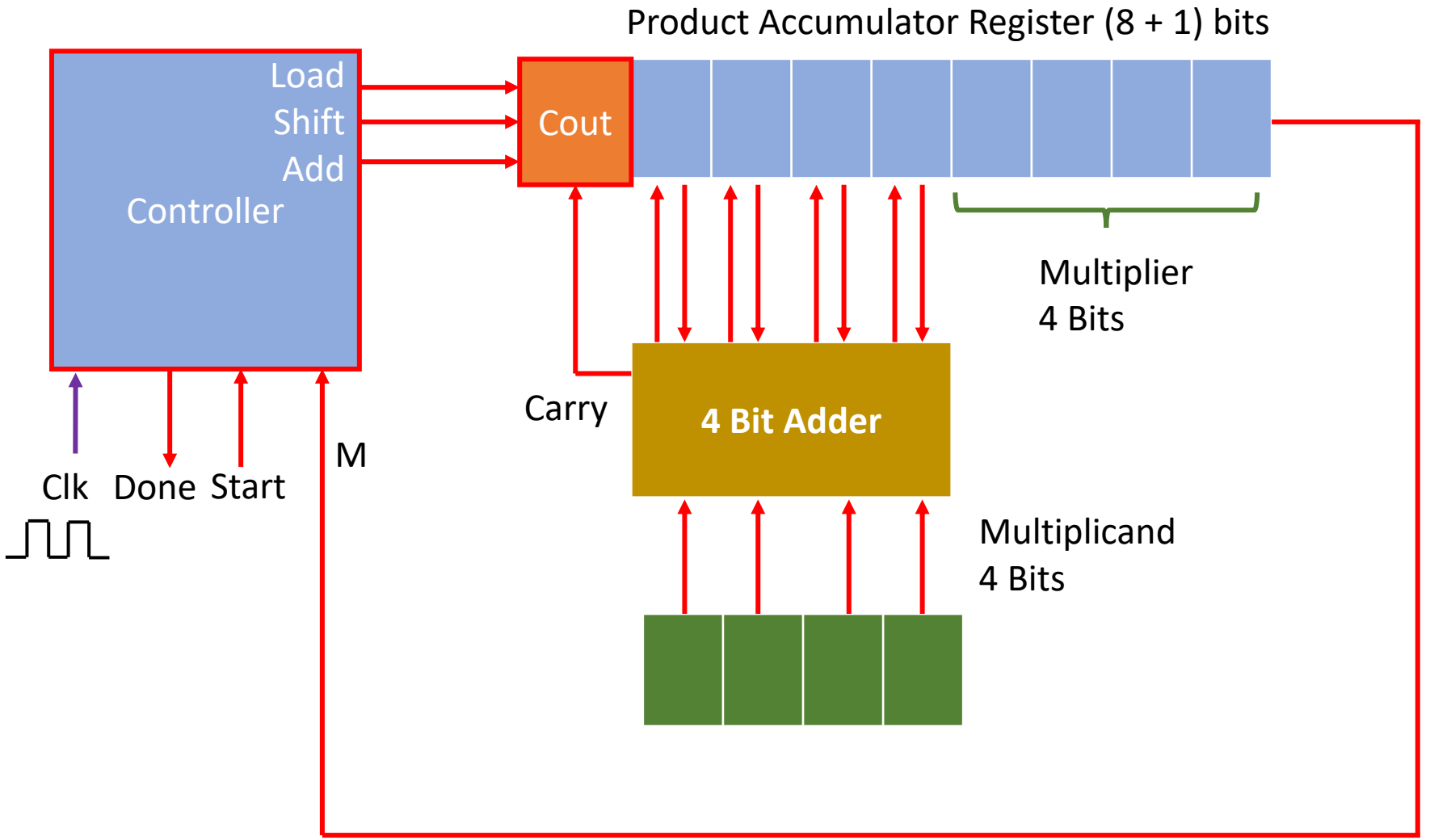


STG of Reduced Register Sequential Multiplier





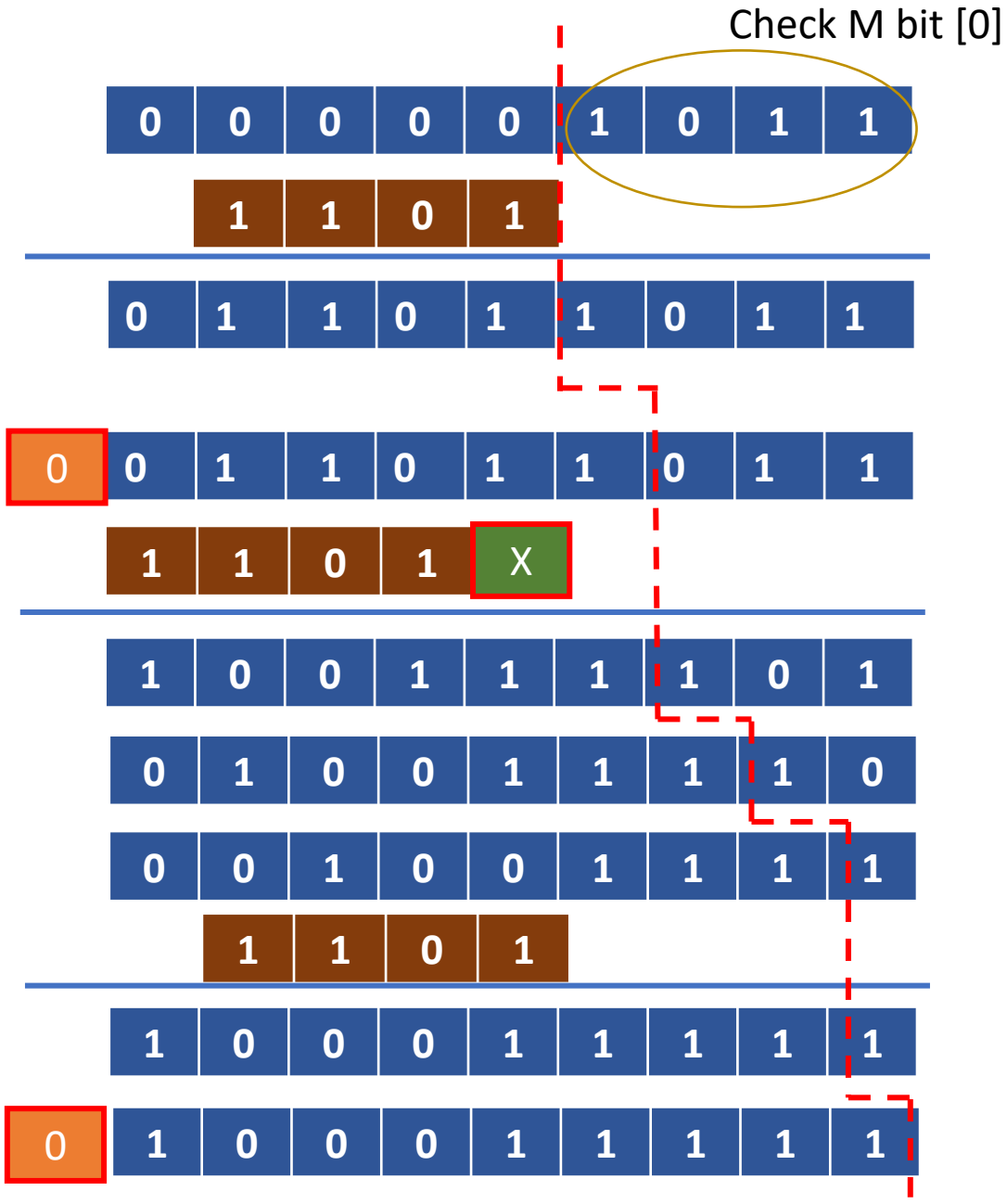
Example of a 4-bit Serial Parallel Multiplier



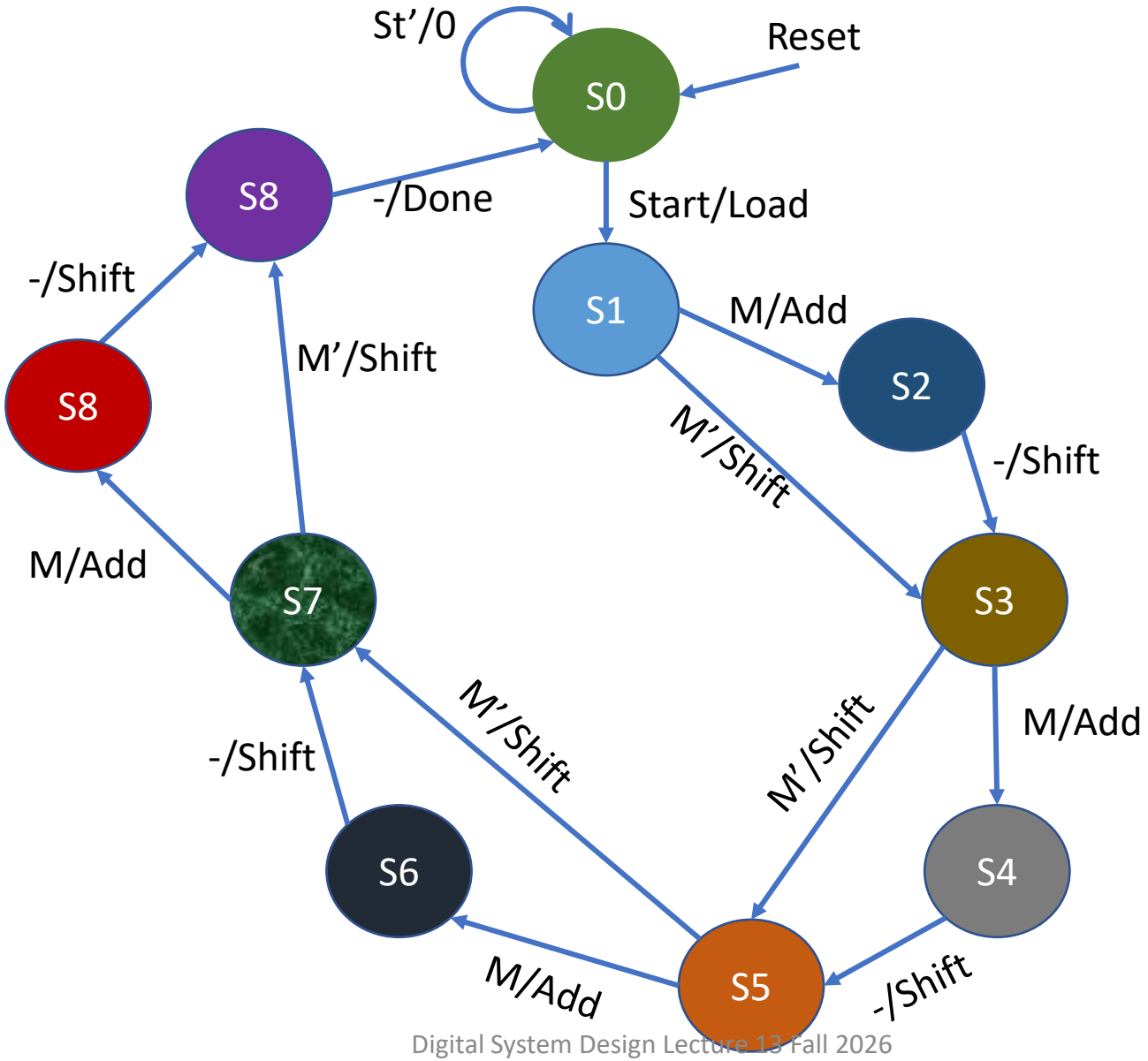
Shift the contents
To the right after
Every step

Multiplier Register Operation

Initial Contents of Accumulator
Multiplicand bit [0] is ‘1’
After Add operation
After Shift Right
Next bit M = 1 hence Add
After Add
After Shift Right
Next bit M=0, hence Skip Add operation
After Shift Right
Next bit M=1, hence Add
After Addition
After Shift Right, final answer

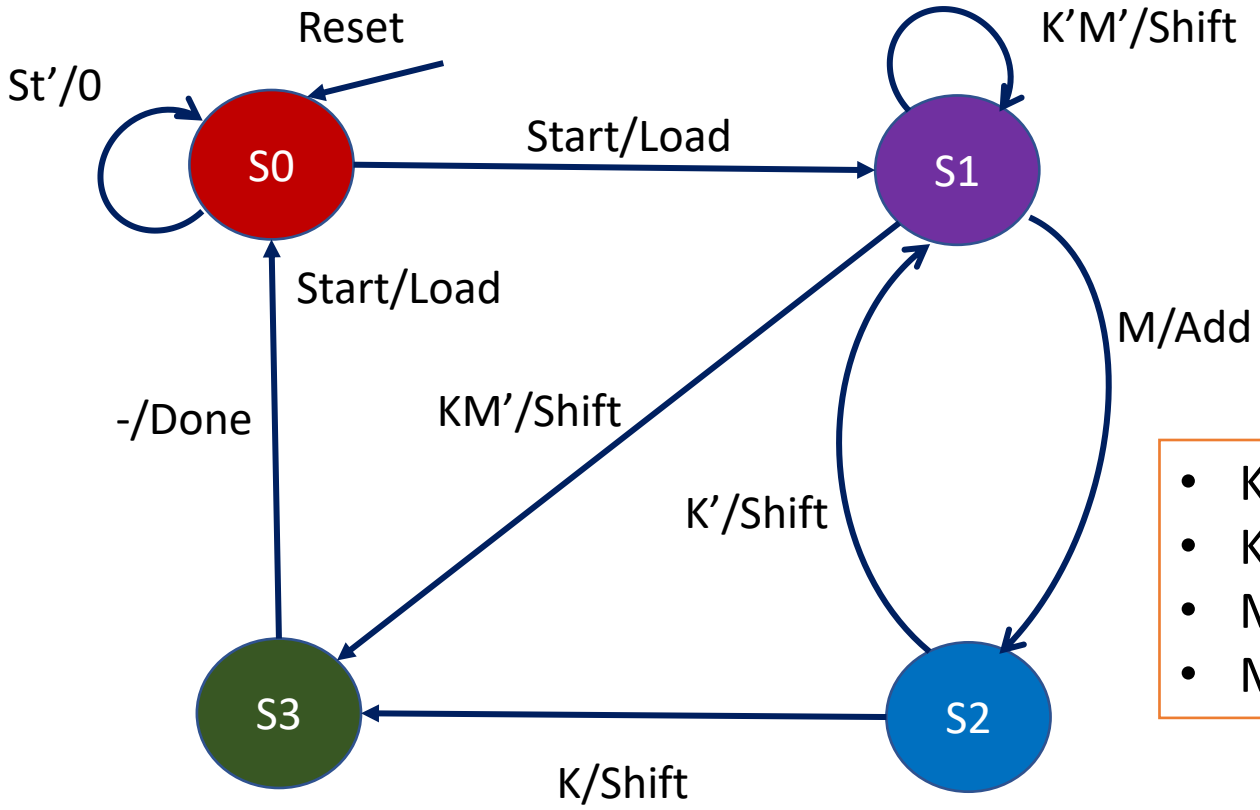


STG Control Diagram for this Multiplier



M = relevant bit of multiplier

Flexible STG for any no. of multiplicand bits



- K = Completion signal generated after (n-1) shifts
- K=1 means one more add (if needed) and shift operation
- M=0 means do the final shift
- M=1 means add before shift and go to S2

Signed and Fractional Multiplication

Multiplication of Signed Binary Numbers

Case I: Negative Multiplicand, Positive Multiplier

Example: $-3_{10} \times 6_{10}$

Sign-bit of the multiplicand must be extended to the word length of the final product before Operating on the 2's Complement words.

This sign-extended multiplicand is used when forming Partial products and accumulated sums.

The result of the multiplication is the 2's Complement of the Product. The final magnitude is found by taking 2's Complement.

Bit Assignment: We assign 8 bits to both numbers. The product will thus be 16 bits.
+3 = 0000 0011
Thus 2's Complement = -3 = 1111 1101
+6 = 0000 0110

1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1
							x	0	0	0	0	0	1	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	X
1	1	1	1	1	1	1	1	1	1	1	1	0	1	X	X
1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	0

Remember: Sign Extension to maximum number of bits in datapath

Answer = (1111 1111 1111 10 1110)
Take 2's Complement, Answer = -(010010) = -18_{10}

Multiplication of Fractions

Convert from decimal to binary $(\frac{3}{4})$

= 0.75

0.75 x 2 = 1.5, keep 1

0.5 x 2 = 1.0, keep 1

0 x 2 = 0 keep 0

And only zeros afterwards

= $2^{-1} + 2^{-2} + 0 + 0$

= 0.1100; assigning four fractional bits

2's Complement of Binary Fractional Nos.

- Given binary fractional number = (0.1100)

- Method 1:
- Decide on the number of total bits, eg. 5 bits; Invert all bits; add +1 to LSB

- 2's Complement = 1.0011
- + 1
- _____
- = 1.0100
- Method 2:
- Look from right to left; when you Encounter first 1; invert all bits to the left

- For (0.1100), the 2's Complement = (1.0100)

Question?

- Represent 9/16 using five fractional bits:

- Hint: 9/16 = 0.xxxxx
- Keep multiplying by 2; if answer is greater than 1, keep 1

- Solve: 0.562 x 2 = 1.125, keep 1
- 0.125 x 2 = 0.25, keep 0
- 0.25 x 2 = 0.5, keep 0
- 0.5 x 2 = 1.0, keep 1
- 0 x 2 = 0, keep 0, and same for more terms
- Answer = 0.10010 in binary

Finally, 2's Complement of "0.10010" is (look right to left) "1.01110" that is (-9/16)

Convert Fraction Number to Binary

- Represent 9/16 using five fractional bits:
- Hint: 9/16 = 0.5625
- Keep multiplying by 2; if answer is greater than 1, keep 1

- Solve: 0.5625 x 2 = **1**.125, keep 1
- 0.125 x 2 = **0**.25, keep 0
- 0.25 x 2 = **0**.5, keep 0
- 0.5 x 2 = **1**.0, keep 1
- 0 x 2 = **0**, keep 0, and same for more terms
- Answer = 0.10010 in binary

Finally, 2's Complement of "0.10010" is (look right to left) "1.01110" that is (-9/16)

Multiplication of Signed Fractions

- Fractions are multiplied like whole numbers, but overflow is not possible
- A 4-bit fractional number is represented as minimum 5-bit fixed point number with MSB holding the sign bit in 2's Complement format
- The product of two 5-bit numbers will produce 10-bit result
- MSB will be sign-extended (bit replication) for negative multiplicand

Example 1: Positive multiplicand, positive multiplier, fraction multiplication

Show binary multiplication of $(3/4)_{10}$ x $(1/2)_{10}$
Use 5-bits to represent each number

3/4 = 0.1100

1/2 = 0.1000

					0.	**1**	**1**	**0**	**0**	Positive multiplicand
				x	**0.**	**1**	**0**	**0**	**0**	Positive multiplier
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	X	0
0	0	0	0	0	0	0	0	X	X	0
0	0	0	1	1	0	0	X	X	X	1
0	0.	0	1	1	0	0	0	0	0	

Check Multiplier bits one at a time

Insert decimal, count how many bits after decimal in both numbers (4 + 4 = 8 bits in both numbers)

Answer = (0.0110000000) = +(3/8)

Example 2: Negative multiplicand, positive multiplier, fraction multiplication

Show binary multiplication of $(-3/4)_{10}$ x $(3/8)_{10}$
Use 5-bits to represent each number

3/4 = 0.1100
-3/4 = -(0.1100)
= 1.0100

3/8 = 0.0110

Sign bit replication

		1	**1**	**1**	**1**	**1**	**1.**	**0**	**1**	**0**	**0**
						x	**0.**	**0**	**1**	**1**	**0**
	1	1	1	1	1	1	0	1	0	0	X
1	1	1	1	1	1	0	1	0	0	X	X
Extra	1	1.	1	0	1	1	1	0	0	0	

Negative multiplicand

Positive multiplier
(first 0 only shift, then 1, Add multiplicand)
(next 1, Add Multiplicand, then 0 shifts only)

Insert decimal, count how many bits after decimal in both numbers (4 + 4 = 8 bits in both numbers)
Answer = (11.10111000) , take 2's complement = -(0.01001000) = (-9/32)

Example 3: Positive multiplicand, negative multiplier, fraction multiplication

Show binary multiplication of $(3/4)_{10}$ x $(-3/8)_{10}$
Use 5-bits to represent each number

3/4 = 0.1100

3/8 = 0.0110
-3/8 = -(0.0110)
= (1.1010)

				0.	**1**	**1**	**0**	**0**	
				x	**1.**	**1**	**0**	**1**	**0**
				0	0	0	0	0	
			0	1	1	0	0	X	
		0	0	0	0	0	X	X	
	0	1	1	0	0	X	X	X	
1	0	1	0	0	X	X	X	X	
1.	1	0	1	1	1	0	0	0	

Positive multiplicand

Negative multiplier

Check Multiplier bits one at a time

2's Complement of Multiplicand due to 1 MSB

Insert decimal, count how many bits after decimal in both numbers (4 + 4 = 8 bits in both numbers)

Answer = (1.10111000), take 2's Complement = -(0.01001000) = -(9/32)

Example 4: Negative multiplicand, negative multiplier, fraction multiplication

Show binary multiplication of $(-3/4)_{10}$ x $(-3/8)_{10}$
Use 5-bits to represent each number

3/4 = 0.1100
-3/4 = -(0.1100)
= 1.0100

3/8 = 0.0110
-3/8 = -(0.0110)
= 1.1010

Sign bit replication

					1	1	1	1	1	1.	0	1	0	0	Negative multiplicand
									x	1.	1	0	1	0	Negative multiplier
				Extra ←	0	0	0	0	0	0	0	0	0	0	
				1	1	1	1	1	1	0	1	0	0	X	
			1	0	0	0	0	0	0	0	0	0	X	X	
			1	1	1	1	1	1	0	1	0	0	X	X	X
			0	0	0	0	0	1	1	0	0	X	X	X	X
		1	0	1	0	0	0.	0	1	0	0	1	0	0	0

2's Complement of Multiplicand due to 1 MSB

Insert decimal, count how many bits after decimal in both numbers (4 + 4 = 8 bits in both numbers)

Answer = $+(0.01001000) = 2^{-2} + 2^{-5} = 0.25 + 0.03125 = 0.28125_{10}$

To Remember in Signed Multiplication

- When Multiplicand is Negative, do a sign extension to cover the possible bit-width of Answer
- When Multiplier is Negative, there is a final 2's Complement Addition Step corresponding to the MSB of multiplier

Algorithmic Improvement in Multipliers

Booth Encoding

Booth Multiplication Process

Recap – Fractional and Signed Multiplication

Multiplication of Fractions

Convert from decimal to binary $(\frac{3}{4})$

= 0.75

0.75 x 2 = 1.5, keep 1

0.5 x 2 = 1.0, keep 1

0 x 2 = 0 keep 0

And only zeros afterwards

= $2^{-1} + 2^{-2} + 0 + 0$

= 0.1100; assigning four fractional bits

2's Complement of Binary Fractional Nos.

- **Given binary fractional number = (0.1100)**

- **Method 1:**
- **Decide on the number of total bits, eg. 5 bits; Invert all bits; add +1 to LSB**

- **2's Complement = 1.0011**
- **+ 1**
- **_____**
- **= 1.0100**
- **Method 2:**
- **Look from right to left; when you Encounter first 1; invert all bits to the left**

- **For (0.1100), the 2's Complement = (1.0100)**

Question?

- Represent 9/16 using five fractional bits:

- Hint: 9/16 = 0.xxxxx
- Keep multiplying by 2; if answer is greater than 1, keep 1

- Solve: 0.562 x 2 = 1.125, keep 1
- 0.125 x 2 = 0.25, keep 0
- 0.25 x 2 = 0.5, keep 0
- 0.5 x 2 = 1.0, keep 1
- 0 x 2 = 0, keep 0, and same for more terms
- Answer = 0.10010 in binary

Finally, 2's Complement of "0.10010" is (look right to left) "1.01110" that is (-9/16)

Convert Fraction Number to Binary

- Represent 9/16 using five fractional bits:
- Hint: 9/16 = 0.5625
- Keep multiplying by 2; if answer is greater than 1, keep 1

- Solve: 0.5625 x 2 = **1**.125, keep 1
- 0.125 x 2 = **0**.25, keep 0
- 0.25 x 2 = **0**.5, keep 0
- 0.5 x 2 = **1**.0, keep 1
- 0 x 2 = **0**, keep 0, and same for more terms
- Answer = 0.10010 in binary

Finally, 2's Complement of "0.10010" is (look right to left) "1.01110" that is (-9/16)

Multiplication of Signed Fractions

- Fractions are multiplied like whole numbers, but overflow is not possible
- A 4-bit fractional number is represented as minimum 5-bit fixed point number with MSB holding the sign bit in 2's Complement format
- The product of two 5-bit numbers will produce 10-bit result
- MSB will be sign-extended (bit replication) for negative multiplicand

Example 1: Positive multiplicand, positive multiplier, fraction multiplication

Show binary multiplication of $(3/4)_{10}$ x $(1/2)_{10}$
Use 5-bits to represent each number

3/4 = 0.1100

1/2 = 0.1000

					0.	**1**	**1**	**0**	**0**	Positive multiplicand
				x	**0.**	**1**	**0**	**0**	**0**	Positive multiplier
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	X	0
0	0	0	0	0	0	0	0	X	X	0
0	0	0	1	1	0	0	X	X	X	1
0	0.	0	1	1	0	0	0	0	0	

Check Multiplier bits one at a time

Insert decimal, count how many bits after decimal in both numbers (4 + 4 = 8 bits in both numbers)

Answer = (0.0110000000) = +(3/8)

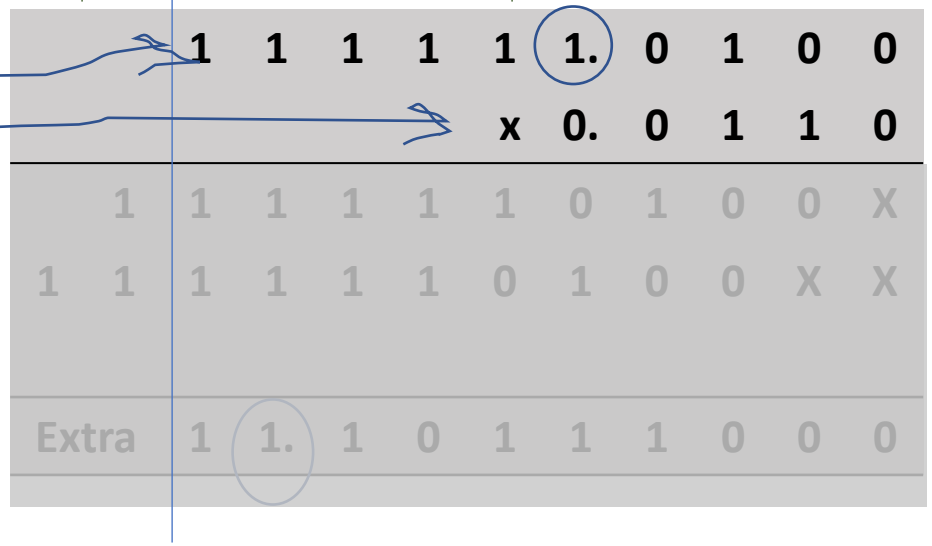
Example 2: Negative multiplicand, positive multiplier, fraction multiplication

Show binary multiplication of $(-3/4)_{10}$ x $(3/8)_{10}$
Use 5-bits to represent each number

3/4 = 0.1100
-3/4 = -(0.1100)
= 1.0100

3/8 = 0.0110

Sign bit replication



Negative multiplicand

Positive multiplier

(first 0 only shift, then 1, Add multiplicand)

(next 1, Add Multiplicand, then 0 shifts only)

Insert decimal, count how many bits after decimal in both numbers (4 + 4 = 8 bits in both numbers)

Answer = (11.10111000) , take 2's complement = -(0.01001000) = (-9/32)

Example 3: Positive multiplicand, negative multiplier, fraction multiplication

Show binary multiplication of $(3/4)_{10}$ x $(-3/8)_{10}$
Use 5-bits to represent each number

3/4 = 0.1100

3/8 = 0.0110
-3/8 = -(0.0110)
= (1.1010)

				0.	**1**	**1**	**0**	**0**	
				x	**1.**	**1**	**0**	**1**	**0**
				0	0	0	0	0	
			0	1	1	0	0	X	
		0	0	0	0	0	X	X	
	0	1	1	0	0	X	X	X	
1	0	1	0	0	X	X	X	X	
1.	1	0	1	1	1	0	0	0	

Positive multiplicand

Negative multiplier

Check Multiplier bits one at a time

2's Complement of Multiplicand due to 1 MSB

Insert decimal, count how many bits after decimal in both numbers (4 + 4 = 8 bits in both numbers)

Answer = (1.10111000), take 2's Complement = -(0.01001000) = -(9/32)

Example 4: Negative multiplicand, negative multiplier, fraction multiplication

Show binary multiplication of $(-3/4)_{10}$ x $(-3/8)_{10}$
Use 5-bits to represent each number

3/4 = 0.1100
-3/4 = -(0.1100)
= 1.0100

3/8 = 0.0110
-3/8 = -(0.0110)
= 1.1010

Sign bit replication

Negative multiplicand

Negative multiplier

2's Complement of Multiplicand due to 1 MSB

					1	1	1	1	1.	0	1	0	0
								x	1.	1	0	1	0
				Extra ←	0	0	0	0	0	0	0	0	0
				1	1	1	1	1	1	0	1	0	0
		1	0	0	0	0	0	0	0	0	0	X	X
		1	1	1	1	1	1	0	1	0	0	X	X
		0	0	0	0	0	1	1	0	0	X	X	X
1	0	1	0	0	0.	0	1	0	0	1	0	0	0

Insert decimal, count how many bits after decimal in both numbers (4 + 4 = 8 bits in both numbers)

Answer = $+(0.01001000) = 2^{-2} + 2^{-5} = 0.25 + 0.03125 = 0.28125_{10}$

To Remember in Signed Multiplication

- When Multiplicand is Negative, do a sign extension to cover the possible bit-width of Answer
- When Multiplier is Negative, there is a final 2's Complement Addition Step corresponding to the MSB of multiplier

Algorithmic Improvement in Multipliers

Booth Encoding

Booth Multiplication Process

Booth Encoded Multipliers

Object: To reduce the number of 'Add' steps required in complete multiplication cycle

MSB '1' shows negative number

2's Complement of $7_{10} = (1\ 0\ 0\ 1)_2$

$1 \times 2^0 = 1$

$0 \times 2^1 = 0$

$0 \times 2^2 = 0$

$-1 \times 2^3 = -8$

Allow both +ive and –ive signs to be used in conversion

Decimal value of $(1001)_2$ = (-8+1) = -7 = **(1 0 0 1) or (-1 0 0 1)**

Booth's algorithm is valid for both positive and negative numbers in 2's complement format

Booth Recoding of a 2's Complement Number

Check two bits

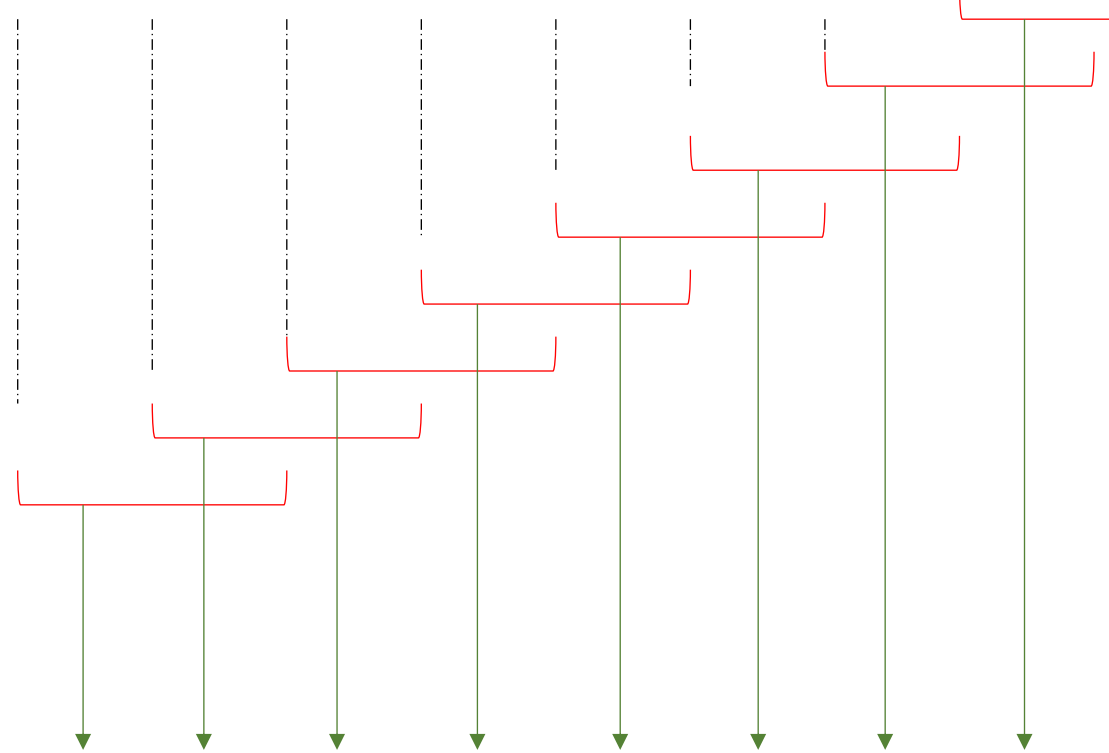
m_i	m_{i-1}	Booth Recoded C_i	Value	Status
0	0	0	0	String of 0s
0	1	1	+1	End of string of 1s
1	0	$\bar{1}$	-1 or $\bar{1}$	Begin string of 1s
1	1	0	0	Midstring of 1s

Booth Recoding of -65_{10}

Append '0' on right, if LSB=1

-65_{10} = | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 2's Complement notation

+65 = (01000001)
2's Complement
-65 = (10111111)



m_i	m_{i-1}	Booth Recoded Ci
0	0	0
0	1	1
1	0	$\underline{1}$
1	1	0

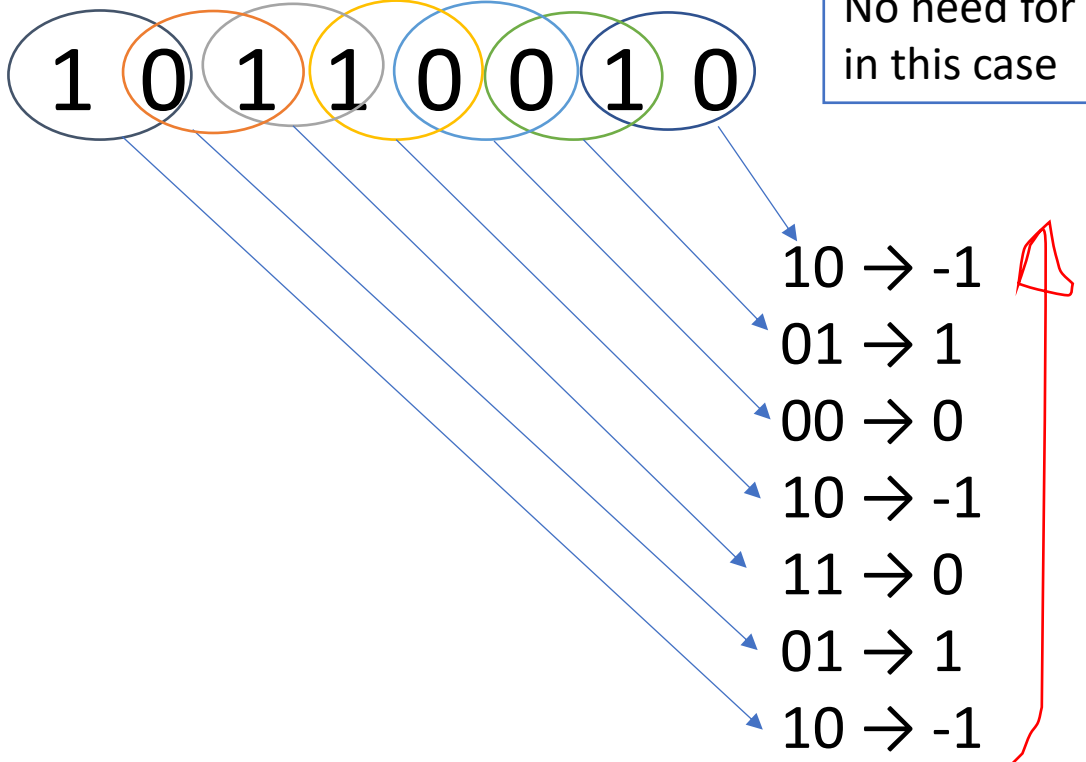
-65_{10} = Or  Booth Recoded notation

Question?

Convert decimal number –78 to Booth Encoded format using 8 binary bits

+78 = 01001110
Take 2's Complement
-78 = 10110010

m_i	m_{i-1}	Booth Recoded Ci
0	0	0
0	1	1
1	0	$\bar{1}$
1	1	0



Answer = -1 1 0 -1 0 1 -1

After Booth Encoding

Booth Encoded Multiplication

Check two bits

m_i	m_{i-1}	Booth Recoded C_i	Decimal Value	Multiplication Action
0	0	0	0	No Operation
0	1	1	+1	Add Multiplicand to Accumulator
1	0	$\bar{1}$	-1 or $\bar{1}$	Subtract Multiplicand from Accumulator (= 2's Complement Add)
1	1	0	0	No Operation

Booth Multiplication – Example 1

Show Booth Encoded multiplication of 6 x 5, using 4 bits for both numbers

6 Multiplicand
x5 Multiplier

Extra	**0**	**0**	**0**	**0**	**0**	**1**	**1**	**0**	
bits	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**1**	**[0]**
	1	1	1	1	1	0	1	0	
0	0	0	0	0	1	1	0	X	
1	1	1	1	1	0	1	0	X	X
0	0	0	0	1	1	0	X	X	X
					X	X	X	X	
0	1	0	0	0	1	1	1	1	0

Imagine Zero bit if LSB = 1

Check 2-bits at a time, Right to Left
Shift Left by 1 after every step

1[0] = Subtract = Add 2's Compl of Multiplicand to Acc

01 = Add Multiplicand to Acc

10 = Subtract = Add 2's Compl of Multiplicand to Acc

01 = Add Multiplicand to Acc

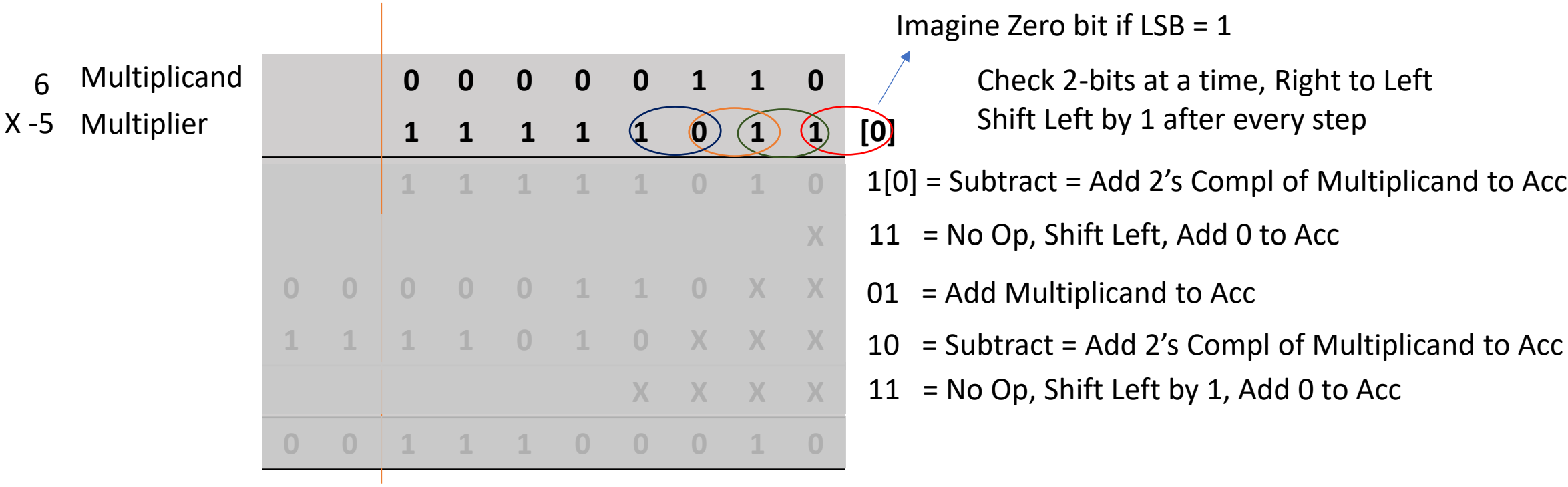
00 = No Op, Shift Left by 1

00 = No Op, Shift Left by 1

Answer = (0001 1110) = +(16 + 14) = $+30_{10}$

Booth Multiplication – Example 2

Show Booth Encoded multiplication of 6 x -5, using 4 bits for both numbers



Answer = (1110 0010) = Take 2's Comp = -(0001 1110) = -30_{10}

Booth Multiplication – Example 1

Show Booth Encoded multiplication of 6 x 5, using 4 bits for both numbers

6 Multiplicand
x5 Multiplier

Extra	**0**	**0**	**0**	**0**	**0**	**1**	**1**	**0**	
bits	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**1**	**[0]**
	1	1	1	1	1	0	1	0	
0	0	0	0	0	1	1	0	X	
1	1	1	1	1	0	1	0	X	X
0	0	0	0	1	1	0	X	X	X
					X	X	X	X	
0	1	0	0	0	1	1	1	1	0

Imagine Zero bit if LSB = 1

Check 2-bits at a time, Right to Left
Shift Left by 1 after every step

1[0] = Subtract = Add 2's Compl of Multiplicand to Acc

01 = Add Multiplicand to Acc

10 = Subtract = Add 2's Compl of Multiplicand to Acc

01 = Add Multiplicand to Acc

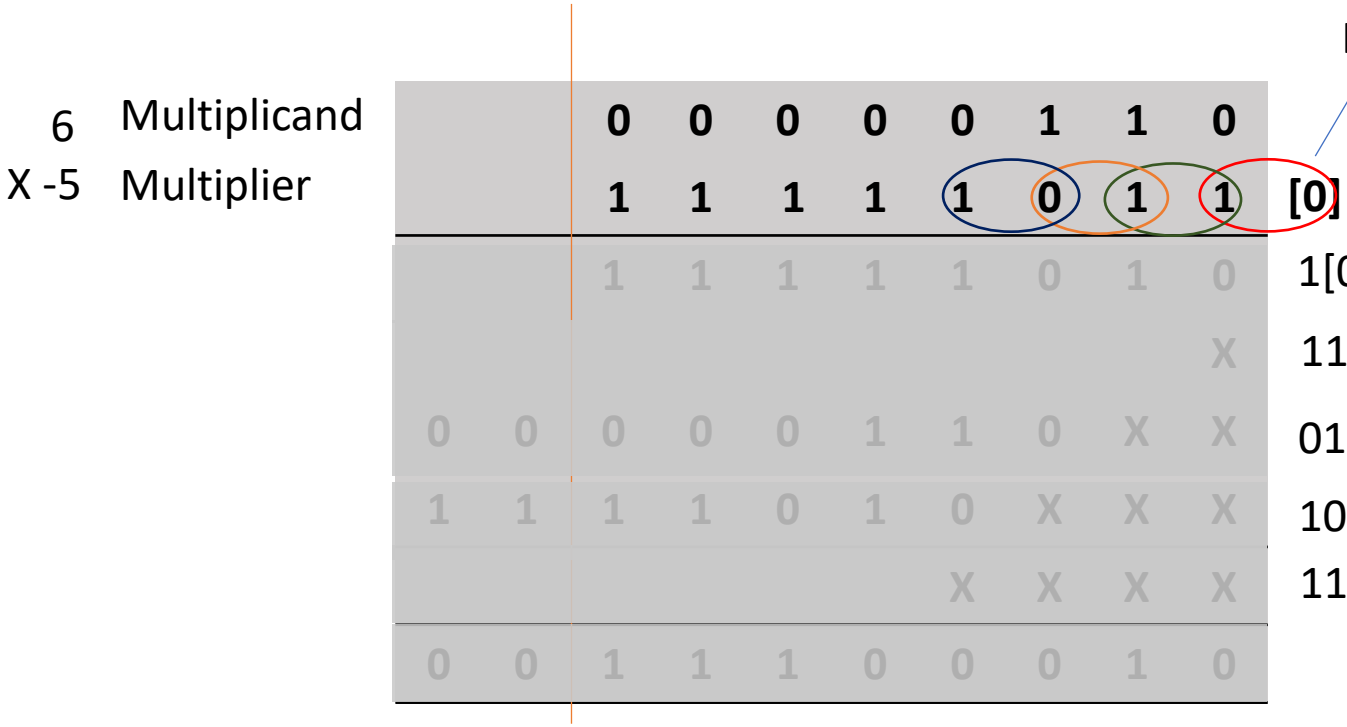
00 = No Op, Shift Left by 1

00 = No Op, Shift Left by 1

Answer = (0001 1110) = +(16 + 14) = $+30_{10}$

Booth Multiplication – Example 2

Show Booth Encoded multiplication of 6 x -5, using 4 bits for both numbers



Imagine Zero bit if LSB = 1

Check 2-bits at a time, Right to Left
Shift Left by 1 after every step

1[0] = Subtract = Add 2's Compl of Multiplicand to Acc

11 = No Op, Shift Left, Add 0 to Acc

01 = Add Multiplicand to Acc

10 = Subtract = Add 2's Compl of Multiplicand to Acc

11 = No Op, Shift Left by 1, Add 0 to Acc

Answer = (1110 0010) = Take 2's Comp = -(0001 1110) = -30_{10}

Booth Multiplication – Example 3

Show Booth Encoded multiplication of B3 x C3, using 8 bits for both numbers

B3 = 1011 0011 = 2's Compl of = 0100 1101 = $(4D)_{16}$ = 77_{10}

C3 = 1100 0011 = 2's Compl of = 0011 1101 = (3D) = 61_{10}

Imaginary Zero bit if LSB = 1

Check 2-bits at a time, Right to Left
Shift Left by 1 after every step

B3	Multiplicand								**1**	**0**	**1**	**1**	**0**	**0**	**1**	**1**
X C3	Multiplier								**1**	**1**	**0**	**0**	**0**	**0**	**1**	**1** **[0]**
		0	0	0	0	0	0	0	0	1	0	0	1	1	0	1
																X
		1	1	1	1	1	1	1	0	1	1	0	0	1	1	X
												X	X	X	X	X
		0	0	0	1	0	0	1	1	0	1	X	X	X	X	X
1		0	0	0	1	0	0	1	0	0	1	0	1	1	0	0

1[0] = Subtract = Add 2's Compl of Multiplicand to Acc

11 = No Op, Shift Left, Add 0 to Acc

01 = Add Multiplicand to Acc

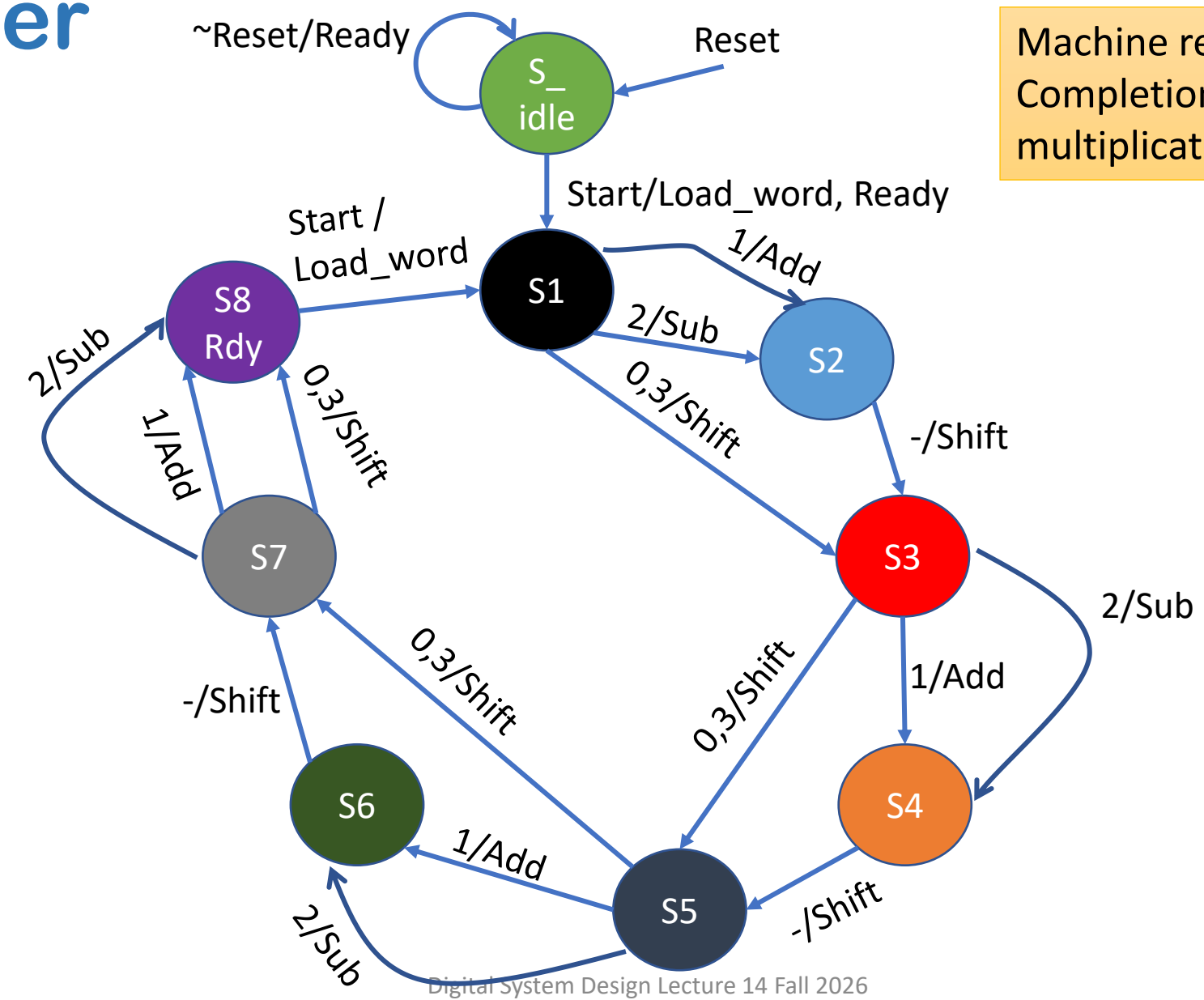
00, 00, 00 = No Op, Shift Left by 1

10 = Subtract = Add 2's Compl of Multiplicand to Acc

11 = No Op, Shift Left, Add 0 to Acc

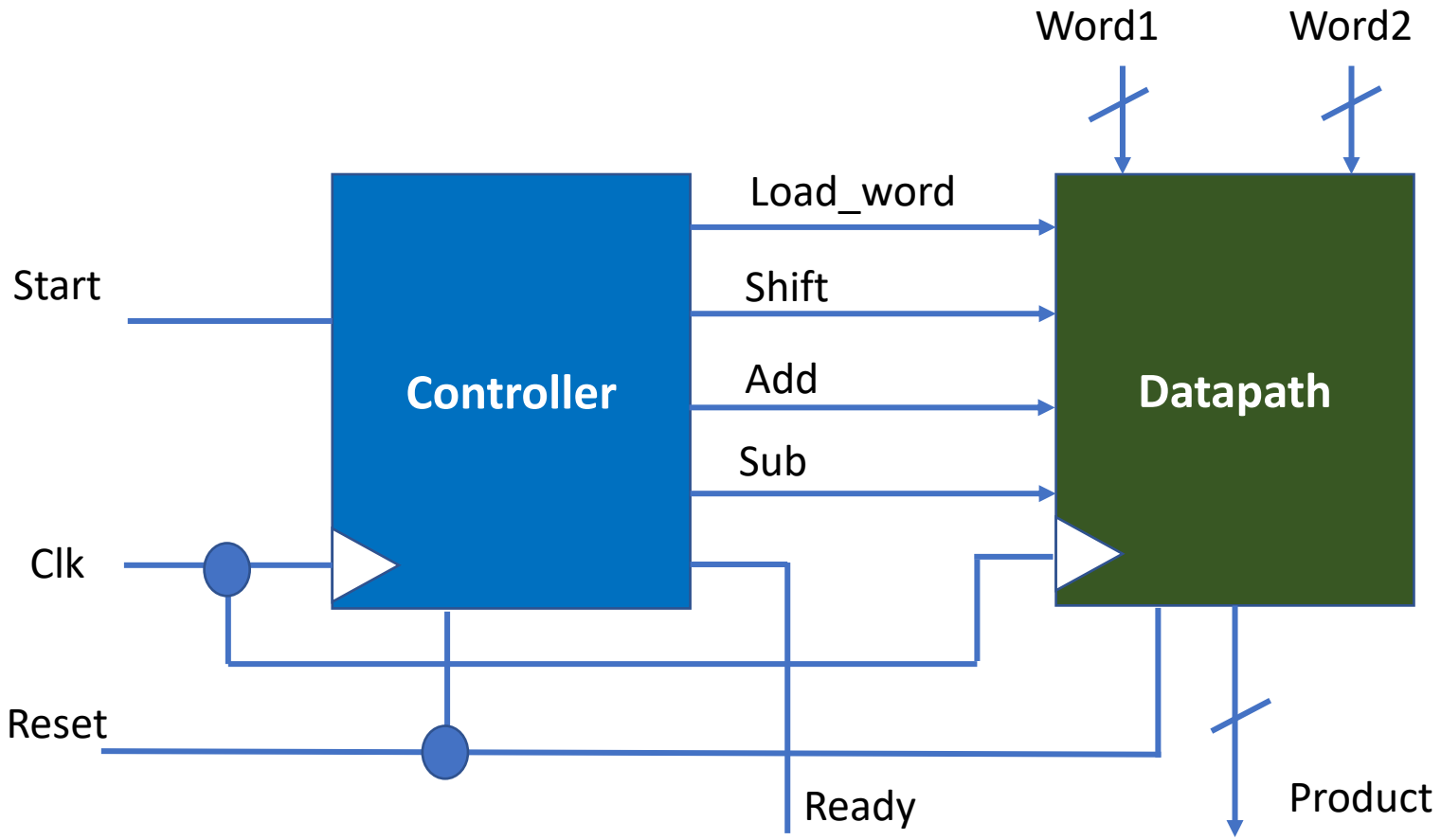
Answer = (0001 0010 0101 1001)2 = (1259)Hex = $(1x16^3 + 2x16^2 + 5x16^1 + 9x16^0) = 4697_{10}$

STG for a 4 Bit Booth Encoded Sequential Multiplier



Machine returns to Idle state after Completion of 4 bit Booth Encoded multiplication

Data Path Architecture of a Booth Sequential Multiplier



Question?

Perform the following multiplication using Booth Encoding.

Multiplicand = 35, Multiplier = 19

How many Adds and Shifts are required in this multiplication?

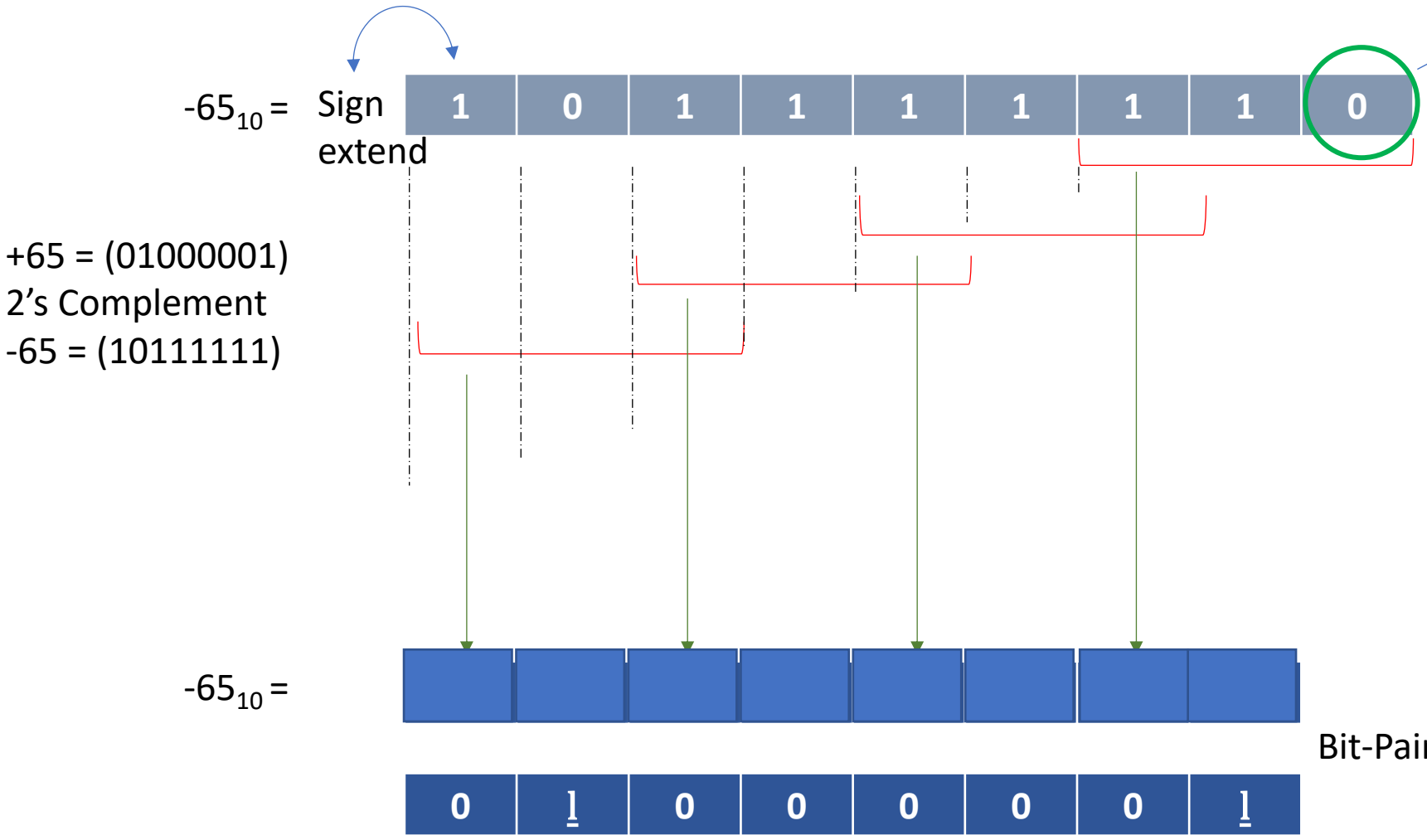
How does this compare to a simple binary array multiplier?

Modified Booth / Radix-4 Multiplication

Bit-Pair Encoding
Modified Booth Encoding
Radix-4 Encoding

Check Three Bits				Coded Two Bits				
m_{i+1}	m_i	m_{i-1}	Decimal	BRC_{i+1}	BRC_i	Value	Status	Multiply Actions
0	0	0	0	0	0	0	String of 0s	Shift by 2
0	0	1	1	0	1	+1	End of string of 1s	Add
0	1	0	2	0	1	+1	Single 1	Add
0	1	1	3	1	0	+2	End of string of 1s	Shift by 1, Add, Shift by 1
1	0	0	4	$\bar{1}$	0	-2	Begin of string of 1s	Shift by 1, Subtract, Shift by 1
1	0	1	5	0	$\bar{1}$	-1	Single 0	Subtract
1	1	0	6	0	$\bar{1}$	-1	Begin of string of 1s	Subtract
1	1	1	7	0	0	0	Midstring of 1s	Shift by 2

Bit-Pair / Radix-4 Recoding of -65_{10}



Check 3 bits			Coded 2 bits		
m_{i+1}	m_i	m_{i-1}	BRC_{i+1}	BRC_i	Value
0	0	0	0	0	0
0	0	1	0	1	+1
0	1	0	0	1	+1
0	1	1	1	0	+2
1	0	0	1̲	0	-2
1	0	1	0	1̲	-1
1	1	0	0	1̲	-1
1	1	1	0	0	0